

Architectural Specification: Prime Spiral Sieve

DECOUPLED ARCHITECTURE AND ZERO-COPY SERIALIZATION LAYER FOR UOR INTEGRATION

Author: Gary Croft | Target Context: UOR Validation Framework Integration | Implementation Language: Rust (Core Engine)

1. Executive Summary & Design Philosophy

This document defines the formal engineering specification for integrating the **Prime Spiral Sieve** into the **Universal Object Reference (UOR)** framework. The underlying mathematics of the Prime Spiral Sieve leverage structural constraints within $\mathbb{Z}/30\mathbb{Z}$ and $\mathbb{Z}/90\mathbb{Z}$ rings, mapping numbers across a periodic toroidal geometry that reveals deterministic, period-24 digital root cycles and factor dyad symmetries.

Conversely, the UOR architecture operates natively on binary rings, specifically $\mathbb{Z}/2^n\mathbb{Z}$, optimized for binary bitvectors, carry-chain algebra, and 2-adic approximations. Forcing a non-power-of-two modulo domain ($\mathbb{Z}/30\mathbb{Z}$) directly into UOR's internal term grammar creates severe algebraic friction, destroying hardware bit-packing boundaries and imposing heavy compiler and computational overhead.

The Decoupling Principle: To preserve maximum efficiency, the system explicitly separates mathematical execution from data validation and transport. The core sieve engine computes arithmetic entirely within its native primordial domain, outputting a fully flattened, bit-packed stream. The UOR framework ingests this stream as a static, arbitrary data payload, preserving absolute **byte-to-byte correctness** without attempting to compute or bridge non-isomorphic moduli.

2. The Serialization Layer: 8-Bit Modulo 30 Bit-Packing

Within a $\mathbb{Z}/30\mathbb{Z}$ wheel factorization system, all prime numbers greater than 5 are structurally confined to exactly 8 active residue classes: $R_A = \{1, 7, 11, 13, 17, 19, 23, 29\}$. Multiples of 2, 3, and 5 are automatically sieved by the geometry of the wheel.

This allows an elegant 1:1 mapping between one full rotation of the master clock (30 integers) and precisely **1 byte (8 bits)** of memory. Each bit index represents a specific residue class, entirely eliminating memory fragmentation and unaligned bit shifting.

Bit Index	Binary Mask	Residue Class (Mod 30)	Example (Rotation 0: Range 0-29)	Twin Prime Assignment
Bit 0	0x01 (00000001)	+1	1 (Unit)	-
Bit 1	0x02 (00000010)	+7	7	-
Bit 2	0x04 (00000100)	+11	11	Twin Pair A (Lower)
Bit 3	0x08 (00001000)	+13	13	Twin Pair A (Upper)
Bit 4	0x10 (00010000)	+17	17	Twin Pair B (Lower)
Bit 5	0x20 (00100000)	+19	19	Twin Pair B (Upper)
Bit 6	0x40 (01000000)	+23	23	-
Bit 7	0x80 (10000000)	+29	29	-

A bit value of 1 indicates the number at that position remains a verified prime; a value of 0 indicates the position has been eliminated as a composite number. This structure allows the frontend consumer or verification engine to read twin prime distributions natively using trivial hardware bitwise masks (e.g., checking if `byte & 0x0C == 0x0C` or `byte & 0x30 == 0x30`).

3. Modulo 90 Factorization Dyad Core Integration

Instead of using standard, computationally expensive trial division, the core engine utilizes **576 Modulo 90 Factorization Dyads** to systematically cross out composite numbers. The Modulo 90 domain maps across exactly 3 full rotations of the Modulo 30 master clock, split into three distinct matrix tiers:

- **Tier 0:** Residues {1, 7, 11, 13, 17, 19, 23, 29} &mod{90}
- **Tier 1:** Residues {31, 37, 41, 43, 47, 49, 53, 59} &mod{90}
- **Tier 2:** Residues {61, 67, 71, 73, 77, 79, 83, 89} &mod{90}

The master clock tracks these matrix tiers alongside a 24-step digital root sequence index. By knowing which dyad interactions spawn specific composite classes, the engine maps products directly to coordinate indices within the continuous memory block, altering bits inside target bytes without accessing unaligned registers.

4. Rust Reference Implementation

The reference implementation below is written in pure Rust, designed to run with zero external allocations inside the inner loop, maintaining native compliance with UOR's verification environment.

```

pub const ACTIVE_RESIDUES: [u64; 8] = [1, 7, 11, 13, 17, 19, 23, 29];

#[inline(always)]
pub fn absolute_to_sieve_coordinates(n: u64) -> (u64, u8) {
    let rotation_index = n / 30;
    let residue_mod_30 = n % 30;

    let bit_index = match residue_mod_30 {
        1 => 0, 7 => 1, 11 => 2, 13 => 3,
        17 => 4, 19 => 5, 23 => 6, 29 => 7,
        _ => unreachable!("Dyad logic confines composites strictly to active residues"),
    };
    (rotation_index, bit_index)
}

#[inline(always)]
fn next_structural_factor(current: u64) -> u64 {
    let rem = current % 30;
    let step = match rem {
        1 => 6, 7 => 4, 11 => 2, 13 => 4,
        17 => 2, 19 => 4, 23 => 6, 29 => 2,
        _ => 1,
    };
    current + step
}

pub struct ToroidalSieveEngine {
    pub num_rotations: usize,
}

impl ToroidalSieveEngine {
    pub fn new(num_rotations: usize) -> Self {
        Self { num_rotations }
    }

    pub fn execute_dyad_sieve(&self) -> Vec {
        // Initialize all active bits to 1 (Assume Prime)
        let mut sieve_buffer = vec![0xFFu8; self.num_rotations];

        // Zero out the unit position (1 is not prime)
        sieve_buffer[0] &= !(1 << 0);

        let max_number = (self.num_rotations as u64) * 30;
        let limit_sqrt = (max_number as f64).sqrt() as u64;

        for r in 0..self.num_rotations {
            let rotation_base = (r as u64) * 30;

            for bit_i in 0..8 {
                if (sieve_buffer[r] & (1 << bit_i)) == 0 {
                    continue;
                }

                let p1 = rotation_base + ACTIVE_RESIDUES[bit_i];
                if p1 > limit_sqrt { break; }
                if p1 == 1 { continue; }
            }
        }
    }
}

```

```

        let mut p2 = p1;
        while p1 * p2 < max_number {
            let composite = p1 * p2;
            let (target_rotation, target_bit) =
absolute_to_sieve_coordinates(composite);

                if (target_rotation as usize) < self.num_rotations {
                    sieve_buffer[target_rotation as usize] &= !(1 << target_bit);
                }
                p2 = next_structural_factor(p2);
            }
        }
    }
    sieve_buffer
}
}

```

5. Boundary Interface & Zero-Copy Handshake

To safely transfer data to UOR without initiating memory-cloning overhead, the engine exports a standardized foreign function interface (FFI) boundary. This allows the host validation context to point directly to heap memory initialized by the sieve.

```

#[no_mangle]
pub extern "C" fn fetch_prime_data_blob(num_rotations: usize) -> *mut u8 {
    let engine = ToroidalSieveEngine::new(num_rotations);
    let data_vector = engine.execute_dyad_sieve();

    // Safely transform vector to raw pointer boundary block
    let mut boxed_slice = data_vector.into_boxed_slice();
    let raw_ptr = boxed_slice.as_mut_ptr();

    // Forbid Rust memory management from reclaiming slice before UOR collection
    std::mem::forget(boxed_slice);
    raw_ptr
}

```

Verification Strategy: The output pointer addresses a contiguous sequence of raw bytes, which the UOR validation framework ingests directly as a fixed structural block. The host system treats the input strictly as serialized truth data, establishing byte-to-byte correctness across all hardware targets.